The Relay

Yash Dagar. Code, raw results and conversation: `https://github.com/yashdagar/the-relay`

1. What I built and why

A Python asyncio pipeline. **Understand** (gpt-oss-20b) checks the question is about the papers and splits two-part questions into paper-scoped searches. **Retrieve** is a local BM25 tool over 300-word pieces tagged with paper, section and page. **Decide** refuses with no model call when the best BM25 score is under 7.0. **Draft** (gpt-oss-120b) answers only from the passages, citing passage ids. **Check** has two gates: deterministic (every cited id was retrieved, every number in the answer is in its cited passage) and a model of a different family (qwen3.8-27b). A failed check gets one redraft with specific feedback, then a refusal. A per-model budget governor mirrors Groq's buckets, each request has a 15 s deadline, and under pressure the pipeline steps down: lean draft (gpt-oss-20b, three pieces), then the best passages quoted and marked unchecked, then a budget message. A wrong answer is never a step on that ladder.

The decision that shaped it most: **nothing reaches the person until the check passes.** So first token equals total time in every table. I gave up first-token time (about 0.5 s from the model, about 2 s after the check) so I never retract a sentence, and every deadline is on total time.

2. The real service and my stand-in

Table 1: Limits found (Groq free tier, my key)

Limit	Value found	How I found it	Confidence
Requests per minute	30 per model	40 calls at once on allam-2-7b: 33 ok, 7 got 429 "Limit 30". Docs say 30 for gpt-oss	High (allam), medium
Tokens per minute	8,000 per model	Header on every reply; 4th back-to-back draft got 429 "Limit 8000"	High
Requests per day	1,000 per model	Header; reset grows 86.4 s per request used	High
Tokens per day	200,000 (docs)	Never hit, after about 180K on gpt-oss-120b in 24 h	Low
Requests in flight	None seen up to 40	Same 40-call burst	Medium
What it returns	429 in 91 to 455 ms, JSON naming the limit and the exact wait	Probes	High
Time to recover	TPM 82 ms for 11 tokens; RPM under 5.1 s; RPD 86.4 s per request	429 bodies, headers, polling	Medium

Only running it showed two things: tokens per minute are charged as prompt plus `max_tokens` at admission (78 in, 31 out, charged $142 = 78 + 64$), and `retry-after` rounds up to whole seconds (1 s when the body said 82.5 ms).

Stand-in: a Starlette server on the same streaming API, so switching is a base URL. Per-model minute and day buckets refill continuously, charge prompt plus `max_tokens` and refuse with Groq-shaped 429s. First-token time is log-normal per model, fitted to p50 and p95 of 20 real draft calls each. It can fail, succeed slowly, slow down above 16 calls in flight and stream pieces. For gold questions it answers from the gold evidence, if retrieval sent that passage. **Least like the real thing:** answers are perfect by construction, so stand-in rows measure delivery, not quality; and the slowdown under load is a guess, never triggered because free-tier limits kept calls in flight under 16.

3. Results

Stand-in rows use the measured free-tier limits; the real row is the default-setting Table 3 run.

Table 2a: Latency

Scenario	Real or stand-in	Requests	Duration (s)	Offered (per min)	First token p50 / p95 / p99 (ms)	Total p50 / p95 / p99 (ms)
Steady, 2/min, 30 min	stand-in	71	1781	2.4	1507 / 4727 / 6987	1507 / 4727 / 6987
Burst, 20 in 30 s on 1/min	stand-in	50	1187	2.5	2595 / 13975 / 14457	2595 / 13975 / 14457
Overload, 6/min, 20 min	stand-in	143	1192	7.2	3342 / 12848 / 14207	3342 / 12848 / 14207
Minute limits halved	stand-in	71	1785	2.4	2036 / 10480 / 12126	2036 / 10480 / 12126
Daily budget exhausted	stand-in	50	1095	2.7	6 / 3878 / 6504	6 / 3878 / 6504
Paced 3/min	real	20	382	3.2	1958 / 4199 / 7112	1958 / 4199 / 7112

Table 2b: Outcomes

Scenario	Succeeded %	Rate-limited %	Timed out %	Retries per request	Peak queue	Tokens per answer	Cost per 1,000 answers
Steady	98.6	1.4	0.0	0.00	1	4484	$1.84
Burst	60.0	40.0	0.0	0.00	13	4478	$1.82
Overload	67.1	32.2	0.7	0.01	6	3747	$1.43
Minute limits halved	74.6	25.4	0.0	0.01	1	3893	$1.52
Daily exhausted	34.0	66.0	0.0	0.00	1	4307	$1.79
Paced 3/min (real)	100.0	0.0	0.0	0.00	0	4763	$2.03

"Rate-limited" means the person got the passages or budget reply because a limit was near. The governor waits before sending, so Groq rarely sends a 429 and retries stay near zero. Cost uses Groq's paid list prices per model.

The number I could not have guessed: the check step is **77 to 79% of the bill** in every run. qwen3.8-27b costs \$0.80/\$4.00 per million tokens against \$0.15/\$0.60 for the drafter, and it rereads every passage.

Quality, my 36 questions (25 answerable, 5 not in the papers, 3 off-topic, 3 injections), real Groq: 21 of 25 answerable correct and cited, 1 wrong, 3 wrongly refused. All 8 unanswerable and off-topic refused, and 2 of 3 injections handled; inj-03 was refused because the right reply ("160 ms, not 5 ms") repeats a number the paper lacks, and I keep the number check strict. **Zero wrong answers under any load shape.** About 4,800 tokens per answer.

4. Where the time goes

dc-04 on Groq: 11,395 ms, all before the first token. **Waiting** 9,057 ms, **working** 1,712 ms (understand 388, draft 863, check 461), **retrying** 560 ms (a first draft that wrote "a gain of 6.4 points", computed, not in the paper, so the number check rejected it), 66

ms retrieval and overhead. The redraft waited because the first draft's reservation had spent the minute's token budget. I know this from the per-attempt trace every reply carries (step, model, attempt, queued and working ms, outcome, tokens). By contrast mos-03 (7,840 ms) never waited: Groq took 3,258 ms for a 49-token routing reply that usually takes 537.

5. When a limit is hit

Per minute: the governor holds a request until tokens refill, if that still leaves 3.1 s for draft and check inside 15 s. If not, the lean model drafts from three pieces (24 lean drafts over 143 requests in overload). If not even that, the person gets the top passages marked unchecked and "try again in a minute". So everyone is delayed up to about 13 s, some get a smaller model's answer, nobody is dropped silently. In the burst 20 of 50 got passages and none waited past 14.6 s. **Per day:** the budget is checked before any model call; with no draft or check budget left the reply comes in about 6 ms. If only the router's model is out, routing is skipped.

Retrieval: quality versus cost

Table 3 (20 fixed gold questions per setting, real Groq)

Setting	Questions	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token p50 / p95 (ms)	Total p50 / p95 (ms)
120 words, 8 pieces, BM25, no re-rank	20	75.0	5.0	20.0	4573	1941 / 4302	1941 / 4302
300 words, 5 pieces, BM25, no re-rank	20	75.0	0.0	20.0	4763	1958 / 4199	1958 / 4199
300 words, 5 pieces, hybrid, no re-rank	20	80.0	0.0	20.0	4962	1978 / 4246	1978 / 4246

They differ by one question each, and dc-04 flips between runs on the same passages, so 20 questions cannot separate them. Offline on 27 questions, 300-word BM25 had the best recall (0.944, p99 0.9 ms); a MiniLM re-rank was worse (0.889) and took 1.6 s a search, so I never ran it end to end. I chose **300 words, 5 pieces, BM25**: same answers, no model, no tail. Near the limit I keep retrieval (it is free) and send fewer pieces.

How this library will mislead a naive retriever

1. **One paper keeps naming another.** "Transformer" runs through the H-Net paper, so x-01 (encoder layers in the Transformer versus H-Net) fills every slot from H-Net. I made sub-searches paper-scoped.
2. **The right words in the wrong section.** dc-07 asks what data H-Net trained on; the Chinese-and-code section repeats "data budget" and wins. un-03 (English-to-Romanian BLEU, not in the paper) scores 19.6, above 14 of 25 answerable questions.
3. **Facts split across pieces.** At 120 words dc-01 loses the smoothing module and answers "routing module and downsampler", grounded and wrong. Moshi's Fisher size breaks across a page: "It consists" / "of 2000 hours".

6. Retries

Two retries per request, shared across steps, so one slow request cannot become a storm. A 429 waits the exact time in the body, not the rounded `retry-after`; a per-day 429 is never retried; a 5xx backs off 250 ms, doubling, with jitter; a timeout is retried only if a whole step still fits. **It made things worse** in dc-04: the redraft was right but needed a second full reservation, waited 9 s, and turned a 2 s answer into 11 s.

7. What the person sees

At 3 pm, in about 6 ms: "The assistant has used up today's model budget, so it cannot write a checked answer right now. The budget resets gradually; try again in about 2 minutes." Below it, the three best passages with paper, section and page, under "They are quoted, not checked, and may not answer it." The 2 minutes come from the measured refill of one request per 86.4 s.

8. What I'm unsure about, what I left out, what I learned

Least certain: tokens per day. If it is a hard 200K per model, the check model (about 2,700 tokens an answer) caps us near 75 answers a day and "about 2 minutes" is badly wrong. **Left out on purpose:** a cache for repeated questions, since bursts after an outage repeat themselves but I wanted the uncached numbers; and streaming the draft before the check. **Learned:** when tokens per minute are charged on `max_tokens`, `max_tokens` is a throughput knob. My drafts use about 120 of the 400 I reserve, and that unused 70% is what pushes two-part questions onto the lean path.

Your feedback on Vaani

[Yash to write after visiting `https://www.vaani.dev/`: what stood out, what confused you, one thing you would change.]

How I worked with the assistant

Full conversation in `docs/conversation/`. [Yash: review and rewrite in your words.]

- **Prompts that mattered:** writing tests for every layer before any pipeline code, so each layer had a measurable contract; "hold the draft until the check passes", which defined fast; and recording findings as internal reports even where the brief does not ask, which is where most write-up numbers came from.
- **Something it got wrong that I caught:** [Yash to fill. The zsh and threshold bugs were caught by the assistant itself.]
- **Where I overruled it:** it wanted the stand-in to emulate the Developer plan for bigger samples; I kept the measured free-tier limits with longer runs, the account we actually have.
- **Distinctly mine:** [Yash to write.]